

INF01151 – Sistemas Operacionais II

Aula 24 - Transações

Prof. Alberto Egon Schaeffer Filho

Introdução

- Sequência de operações em objetos que é executada **de forma atômica** pelos servidores que gerenciam tais objetos
- Necessidade de manter objetos (recursos) de um servidor em um **estado consistente** na presença de
 - Falhas por colapso do servidor/cliente
 - Servidor deve garantir que ou a transação executa completamente (commit) ou seus resultados parciais são apagados (abort/rollback)
 - Acessos concorrentes por múltiplos clientes
 - Um cliente não pode observar efeitos parciais de operações feitas por outro cliente (**isolamento**)

Motivação para transações

- Qual a diferença entre transações e mecanismos de sincronização vistos em aula (como mutex, semáforos e monitores)?
 - Mecanismos vistos atendem somente o problema de acessos concorrentes
- Transações atendem 3 problemas:
 - Tratamento de acessos concorrentes (múltiplos clientes)
 - Tratamento de falhas lógicas em caso de objetos persistentes
 - Tratamento de falhas de hardware
- Exemplo clássico (para falhas)
 - Operação bancária para remessa de fundos
 - Envolve 2 sub-operações distintas
 - Retirada da conta de origem
 - Depósito na conta de destino
 - Implementação distribuída deve prever
 - Falha em uma ou outra operação
 - Se falhar, tudo deve ser cancelado; se a 1ª já foi confirmada, desfazer

Transação

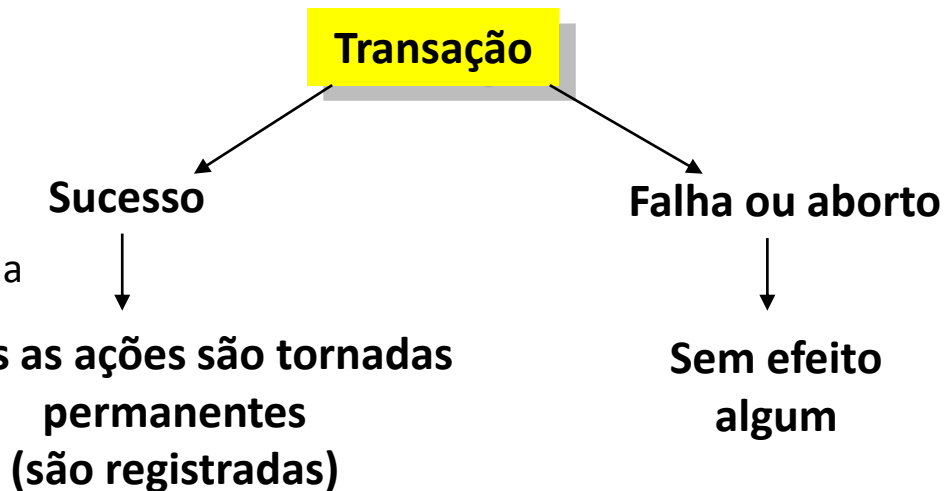
- Em algumas situações, clientes necessitam que uma sequência de requisições independentes sejam **atômicas**
 - Não sofrem interferência de operações executadas por outros processos
 - Ou todas as operações completam com sucesso, ou elas não devem produzir efeito algum na presença de falhas
 - Pode envolver vários servidores → transações distribuídas
 - Pode ser implementada por framework, componente, serviço, middleware, ...
- Propriedades de uma transação
 - **Atômica** (noção de “tudo ou nada”)
 - Todas operações devem ser concluídas com êxito, ou não devem ter nenhum efeito na presença de falhas (ou cancelamento)
 - **Durabilidade**: se transação completar com sucesso, seus “efeitos” são salvos em armazenamento permanente (e.g., disco)
 - **Isolamento**: uma transação não pode observar os efeitos parciais de outras transações (transações disparadas por clientes)

Propriedade A.C.I.D.

Propriedades A.C.I.D. da transação

- Como obter Atomicidade, Isolamento e Durabilidade?
 - Atomicidade e durabilidade → objetos recuperáveis
 - Recuperar estado dos objetos após uma falha do servidor
 - Armazenamento permanente de forma a refletir o efeito de tudo ou nada
 - Isolamento → sincronizar as operações

- Noção de equivalência serial
- Transações devem poder executar de forma concorrente desde que gerem o mesmo resultado de execução serial
- Objetivo é maximizar a concorrência



- E sobre o “C” de “ACID”? **Todas as ações são tornadas permanentes (são registradas)**
 - Consistência
 - Garantir invariantes do sistema
 - Responsabilidade da lógica da transação

Qual o “valor agregado” de transações?

- O que agregam em relação ao já visto (mutex, semáforos, monitores)?
 - Recuperação
 - Na presença de falhas, restaura o estado dos objetos a um valor prévio consistente
 - Maximização de concorrência
 - Transações devem poder executar de forma concorrente

Coordenação de transações

- Transações são criadas e gerenciadas por um coordenador
 - Implementado por um framework, componente, serviço, middleware, etc
- Criação e gerenciamento de cada transação
 - Tem um identificador único (*transaction ID* - TID)
 - Primitivas básicas: *open*, *close* e *abort*
- Implementação:
 - Ao abrir uma transação se obtém um *TID* que deve ser informado a cada operação
 - Transação completa quando cliente chama `closeTransation()` ... ou `abortTransation()`

- `openTransation( )` → TID
 [ação_1\(TID, argumentos\)](#)

 [ação_n\(TID, argumentos\)](#)
- `closeTransaction(TID)`

Opcionalmente:
- `abortTransaction(TID)`

Importante: servidor também pode decidir abortar a transação

Há a noção de efetivação (*commit*) associada ao `closeTransaction`

Exemplos de transação

Operação normal

```
TID = openTransaction( );  
op_1(TID, argumentos);  
op_2(TID, argumentos);  
...  
op_n(TID, argumentos);  
closeTransaction(TID);
```

Abortada pelo cliente

```
TID = openTransaction( );  
op_1(TID, argumentos);  
op_2(TID, argumentos);  
...  
op_n(TID, argumentos);  
abortTransaction(TID);
```

Abortada pelo servidor

```
TID = openTransaction( );  
op_1(TID, argumentos);  
op_2(TID, argumentos);  
...  
op_n(TID, argumentos);  
→ ERRO na operação  
→ Notifica o cliente
```

Informa ao cliente que deve tomar
uma ação apropriada

Fases de uma transação

Fase 1:

Operações são realizadas e registradas de forma a permitir que sejam confirmadas ou desfeitas.

Fase 2:

Modificações decorrentes das operações são **concretizadas** ou **desfeitas**.

openTransaction()



Execução de operações que fazem parte da transação



closeTransaction() OU

abortTransaction()



Commit()

Modificações realizadas são tornadas permanentes



Abort()

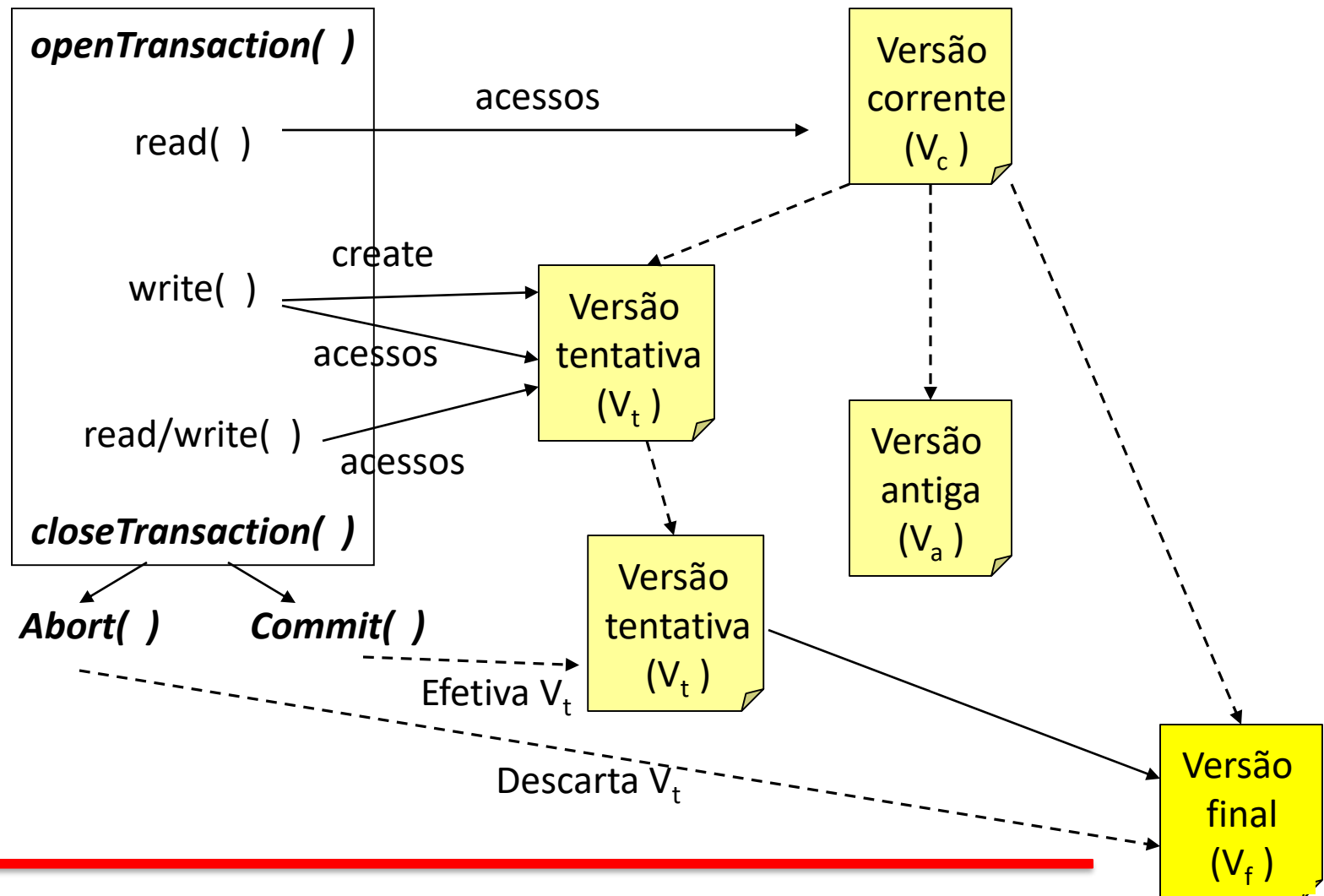
Modificações realizadas não são concretizadas

Abort pode ser feito pelo cliente ou pelo servidor!

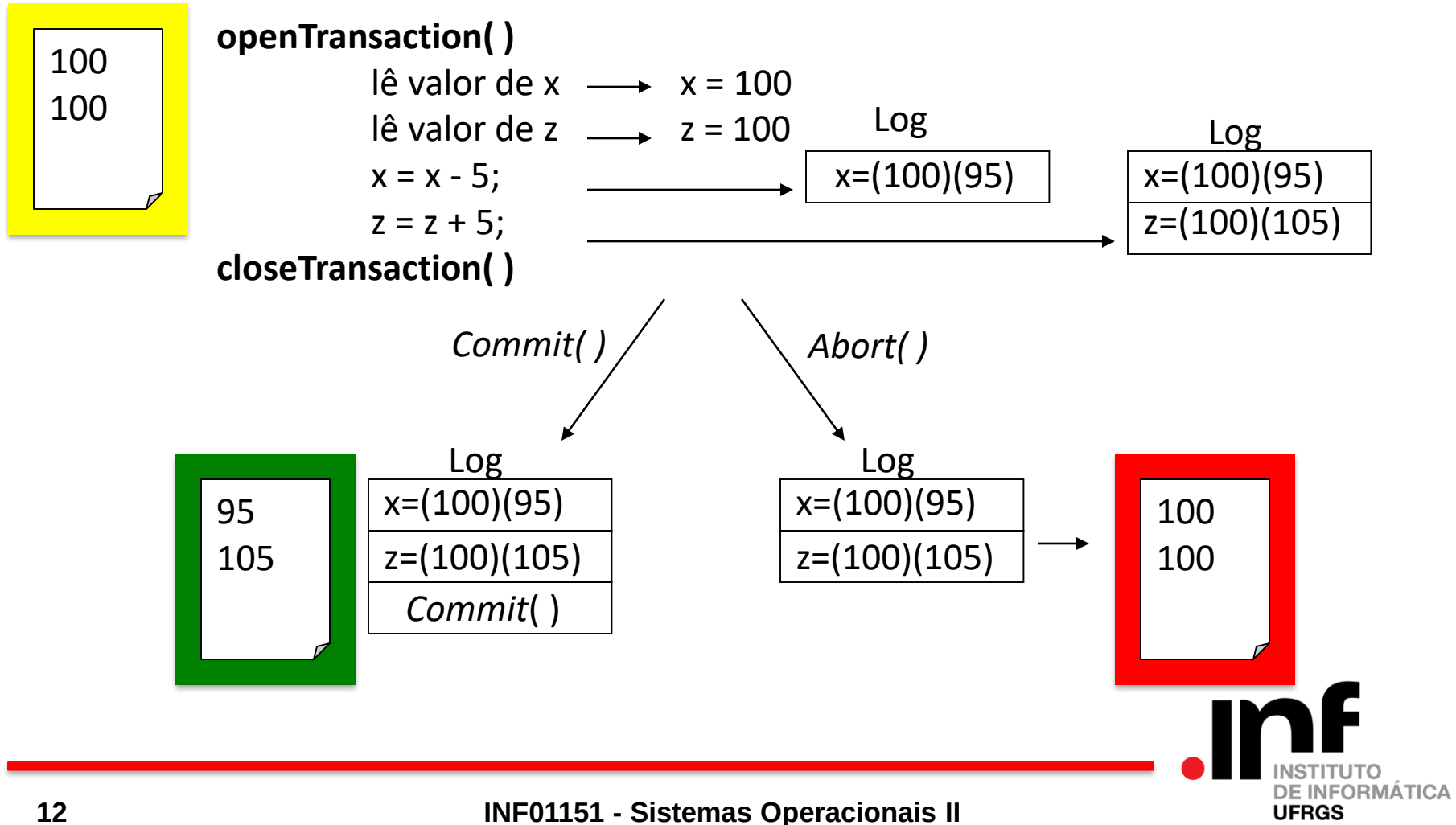
Aspectos de implementação de transações

- Mecanismos de cópias privadas (versionamento)
- Técnica de journalização (logs)

Cópias privadas (versionamento)



Técnica de journalização (*logs*)



Falhas durante transações

- Processo servidor falha
 - Substituição por um novo processo servidor
 - Aborta transações não confirmadas (ou seja, não “committed”)
 - Restaura o estado dos objetos de transações “committed”
 - Processos clientes recebem exceção por timeout ou são avisados que o TID da transação não é mais válido
 - Cliente abandona ou refaz a transação (geralmente com envolvimento humano)
- Processo cliente falha
 - Transações iniciadas no servidor tem tempo de expiração
 - Transações expiradas são abortadas

Controle da concorrência

- Dois problemas bem conhecidos em transações concorrentes:
 - **Atualizações perdidas:**
 - Ocorre quando duas (ou mais) transações lêem o valor antigo de uma variável e depois o utilizam para calcular um novo valor
 - Similar a problema de incremento concorrente duplo em variável compartilhada
 - ✓ Causa: leitura concorrente de valor anterior
 - **Consulta inconsistente**
 - Quando uma transação de consulta (leitura) é executada concorrentemente com uma transação de atualização (escrita)
 - ✓ Causa: série de leituras concorrentes a escritas

Exemplo: transações bancárias (API)

- Operações da interface *conta* (*account*)
 - **deposit(amount)**
 - Deposita **amount** na conta
 - **withdraw(amount)**
 - Retira **amount** da conta
 - **getBalance()** -> **amount**
 - Retorna o saldo da conta: **amount**
 - **setBalance(amount)**
 - Altera o saldo da conta; novo valor = **amount**

Exemplo: transações bancárias (API)

- Operações da interface *agência* (*branch*)
 - **create(name) -> account**
 - Cria nova conta com nome **name**
 - **lookUp(name) -> account**
 - Retorna uma referência a uma conta de nome **name**
 - **branchTotal() -> amount**
 - Retorna o saldo acumulado de todas as contas da agência

Problema da atualização perdida

Condição inicial: conta A = \$100; Conta B = \$200 e Conta C = \$300

Problema: realizar saques nas contas A e C de forma a creditar 10% da conta B na própria (valor final de B = \$ 242,00; valor final de A = \$ 80; valor final de C=\$ 278,00)

Transaction <i>T</i> :	Transaction <i>U</i> :
<i>balance = b.getBalance();</i> <i>b.setBalance(balance*1.1);</i> <i>a.withdraw(balance/10)</i>	<i>balance = b.getBalance();</i> <i>b.setBalance(balance*1.1);</i> <i>c.withdraw(balance/10)</i>
<i>balance = b.getBalance();</i> \$200	
	<i>balance = b.getBalance();</i> \$200
	<i>b.setBalance(balance*1.1);</i> \$220
<i>b.setBalance(balance*1.1);</i> \$220	
<i>a.withdraw(balance/10)</i> \$80	
	<i>c.withdraw(balance/10)</i> \$280

Problema de consulta inconsistente

Condição inicial: conta A = \$200; Conta B = \$200 e Total = \$400

Situação: realizar uma transferência de \$100 da conta A para a conta B e obter o valor total depositado na agência

Transaction V:

a.withdraw(100)

b.deposit(100)

a.withdraw(100); \$100

b.deposit(100) \$300

Transaction W:

aBranch.branchTotal()

total = a.getBalance() \$100

total = total+b.getBalance() \$300

total = total+c.getBalance()

•

Equivalência serial

- O que é:
 - Critério para execução concorrente correta, de forma a **evitar os problemas** de atualizações perdidas e recuperações inconsistentes
- Princípio:
 - Se for sabido que cada uma das transações tem o efeito correto quando executada sozinha,
 - Então é possível inferir que se essas transações forem executadas uma por vez, em alguma ordem, o efeito combinado também será correto
 - Intercalação de operações na qual a combinação dos efeitos é o mesmo que como as transações tivessem sido executadas uma de cada vez **em uma determinada ordem**

Resultado final de duas ou mais transações simultâneas deve ser igual ao obtido pela execução sequencial delas (qualquer ordem)

Equivalência serial entre T e U

Transaction T:	Transaction U:
<i>balance = b.getBalance()</i> <i>b.setBalance(balance*1.1)</i> <i>a.withdraw(balance/10)</i>	<i>balance = b.getBalance()</i> <i>b.setBalance(balance*1.1)</i> <i>c.withdraw(balance/10)</i>

balance = b.getBalance() \$200

*b.setBalance(balance*1.1)* \$220

a.withdraw(balance/10) \$80

balance = b.getBalance() \$220

*b.setBalance(balance*1.1)* \$242

c.withdraw(balance/10) \$278

Equivalência serial entre V e W

Transaction V:	Transaction W:
<i>a.withdraw(100);</i> <i>b.deposit(100)</i>	<i>aBranch.branchTotal()</i>

a.withdraw(100); \$100

b.deposit(100) \$300

total = a.getBalance() \$100

total = total+b.getBalance() \$400

total = total+c.getBalance()

...

Operações conflitantes

- Duas operações são ditas conflitantes quando o efeito combinado delas depende da ordem em que são executadas

Operações		Conflito	Motivo
read	read	não	Duas operações de read independem da ordem em que são executadas
read	write	sim	O valor do read depende se é feito antes ou depois de uma escrita
write	write	sim	O valor final depende da ordem em que os writes são feitos

Para equivalência serial é necessário e suficiente que:

Todos os pares de operações conflitantes entre duas transações sejam executados na mesma ordem em todos os objetos que ambas acessam

Operações conflitantes

- Considere duas transações T e U:
 - T: `x=read(i); y=read(j); write(k,x+y);`
 - U: `write(j,10); write(i,20);`
- Os pares de operações conflitantes são os seguintes:
 - `x=read(i)` e `write(i,20);`
 - `y=read(j)` e `write(j,10);`

T	U
	<code>write(j, 10);</code>
<code>x = read(i);</code>	
	<code>write(i, 20);</code>
<code>y = read(j);</code>	
<code>write(k,x+y)</code>	

NÃO é serialmente equivalente

T	U
<code>x = read(i);</code>	
<code>y = read(j);</code>	
	<code>write(j, 10);</code>
	<code>write(i, 20);</code>
<code>write(k,x+y)</code>	

É serialmente equivalente

Estratégias para controle de concorrência

- Travas (*locks*) e bloqueio
 - Transação (TID) obtém direito de acesso exclusivo sobre um objeto
 - Outras transações devem esperar (ou em alguns casos, compartilhar o lock)
 - É a idéia da exclusão mútua considerando ocorrência de falhas
- Controle otimista
 - Transação executa sempre até que necessite ser efetivada (committed)
 - Verifica se ela executou operações conflitantes, em caso afirmativo, cancela a transação
- Indicação de tempo (*timestamp*)
 - Registra o tempo de leitura e de escrita mais recentes de cada objeto
 - Para cada operação, timestamp da transação é comparado com timestamp do objeto
 - para determinar se operação é efetivada, retardada ou rejeitada (cancelada)

**Basicamente todas forçam, de alguma maneira,
a equivalência serial das operações**

Locks

- Princípio básico:
 - Proibir o acesso (read/write) de diferentes processos através de *locks* no recurso
 - Idéia é que seja feito automaticamente pelo sistema de transações
 - Transações precisam adquirir determinado lock antes de prosseguir
 - Por exemplo, lock para “account B”
- Melhorias:
 - Distinguir os *lock* de escrita daqueles de leitura
 - *Read lock* autoriza outros *read locks*
 - *Write lock* proíbe qualquer outro tipo de *lock* (bloqueios)

Problemas com locks

- Locks por demanda podem levar a *deadlocks* ou inconsistências
 - Para prevenir deadlocks
 - Adquirir locks em uma determinada ordem
 - Adquirir todos os locks necessários de uma só vez
 - Para detectar deadlocks
 - Modelados utilizando grafos de alocação de recursos
 - Representação de transações esperando por um determinado lock
 - Identificação de ciclos → Abortar transação!
- Granularidade do *lock*
 - Lock limita severamente a concorrência
 - Arquivo vs. registro vs. objeto
- Overhead
 - Mesmo transações read-only precisam em geral utilizar locks
 - Garantir que dado lido não está sendo modificado por outra transação

Controle otimista

- Utilizado quando chance de problemas de conflito é pequena
 - Não introduz controles para evitar, mas sim para resolver
 - Quando conflito acontece, alguma transação é geralmente abortada
- Baseado em três etapas:
 - **Trabalho:** cada transação executa suas operações sobre uma cópia privada – *tentative version* – dos objetos (versionamento)
 - Quando houver múltiplas transações concorrentes, podem existir múltiplas cópias
 - **Validação:** verifica se modificações ocorridas no objeto são conflitantes com operações feitas por outras transações
 - Se não, transação é válida. Se sim, transação é abortada
 - **Atualização:** corresponde ao *commit*
 - Alterações na versão tentativa são tornadas permanentes

✓ Vantagens (em relação a locks):

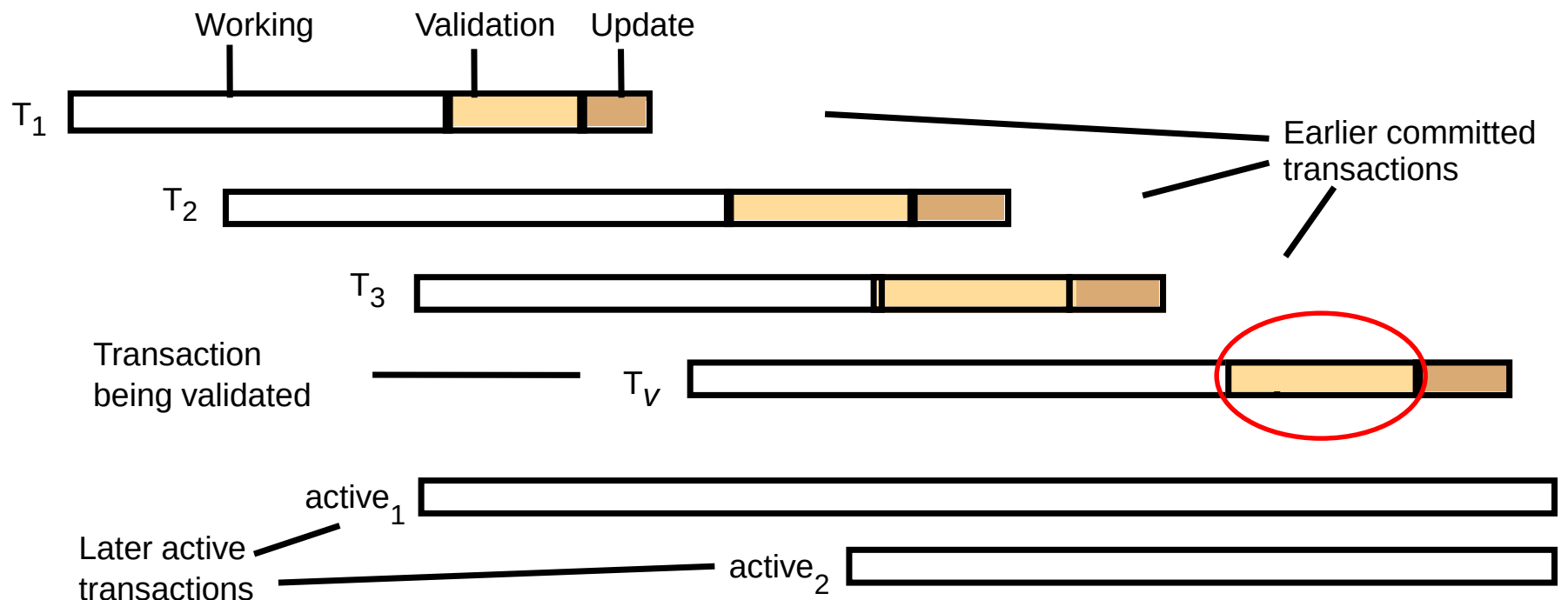
- Livre de deadlock
- Mais paralelismo, menos overhead

✓ Desvantagem:

- “Desfazer” operações em caso de conflito

Controle otimista: validação

- Backward validation vs. Forward validation



Controle otimista: validação

- Backward validation
 - Nenhum dos *reads* de transações mais antigas (e sobrepostas) podem ser afetados
 - Apenas compara *reads* de T_v com *writes* de transações passadas (sobrepostas)

Backward validation of transaction T_v

```
boolean valid = true;
for (int  $T_i$  = startTn+1;  $T_i$  <= finishTn;  $T_i$ ++) {
    if (read set of  $T_v$  intersects write set of  $T_i$ ) valid = false;
}
```

- Forward validation
 - Apenas compara os *writes* de T_v com os *reads* de transações ativas (sobrepostas)
 - Se T_v for transação read-only, sempre passará por essa validação

Forward validation of transaction T_v

```
boolean valid = true;
for (int  $T_{id}$  = active1;  $T_{id}$  <= activeN;  $T_{id}$ ++) {
    if (write set of  $T_v$  intersects read set of  $T_{id}$ ) valid = false;
}
```

Carimbo de tempo (*timestamp*)

- Cada openTransaction recebe um *timestamp*
 - Relógio de Lamport (valores únicos)
 - Define sua posição no tempo na sequência de transações
- Objetos dentro da transação
 - São rastreados com um `read_timestamp` e um `write_timestamp`
 - Cada operação é validada no momento de sua execução
- As transações mais antigas tem preferência sobre as mais novas
 - Em caso de conflito a mais nova é abortada
 - As alterações são desfeitas
 - E usuário eventualmente deve ser avisado

Regra para ordenamento: requisição de escrita em um objeto é válida somente se o objeto foi lido e escrito apenas por transações mais antigas, e requisição para ler um objeto é válida apenas se o objeto foi escrito por transações mais antigas.

Conflito de operações para ordenação de timestamp

Regra	T_c	T_i	
1	escrita	leitura	T_c não deve <i>escrever</i> um objeto que tenha sido <i>lido</i> por qualquer T_i onde $T_i > T_c$; isso exige que $T_c \geq$ a indicação de tempo de leitura máxima do objeto.
2	escrita	escrita	T_c não deve <i>escrever</i> um objeto que tenha sido <i>modificado</i> por qualquer T_i onde $T_i > T_c$; isso exige que $T_c >$ indicação de tempo de escrita do objeto committed.
3	leitura	escrita	T_c não deve <i>ler</i> um objeto que tenha sido <i>modificado</i> por qualquer T_i onde $T_i > T_c$; isso exige que $T_c >$ indicação de tempo de escrita do objeto committed.

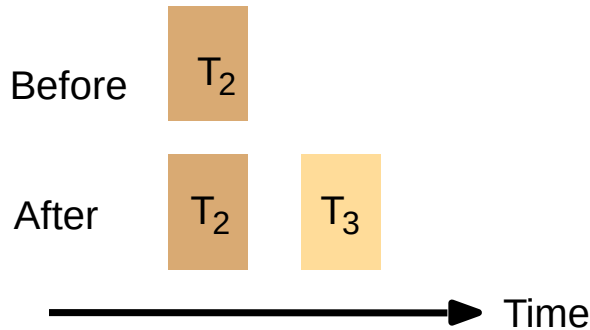
Regra de escrita

Operação de escrita requisitada por transação T_c em objeto D :

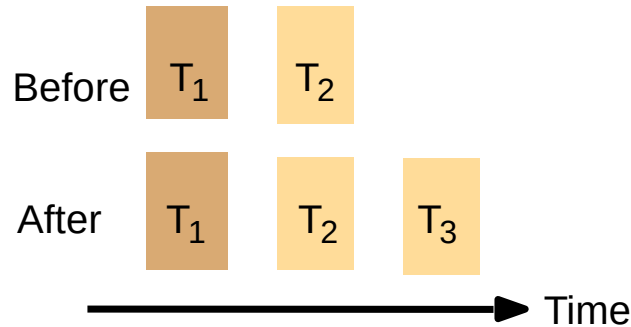
```
if ( $T_c \geq$  maximum read_timestamp on  $D$  &&  
     $T_c >$  write_timestamp on committed version of  $D$ )  
    perform write operation on tentative version  
    of  $D$  with write_timestamp  $T_c$   
else  
    /* write is too late */  
    Abort transaction  $T_c$ 
```


Exemplo de escrita: com $T_3 \geq T_{\text{leitura}}$

(a) T_3 write



(b) T_3 write



Key:

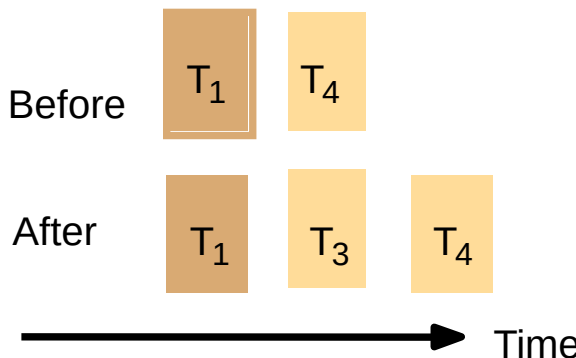


Committed

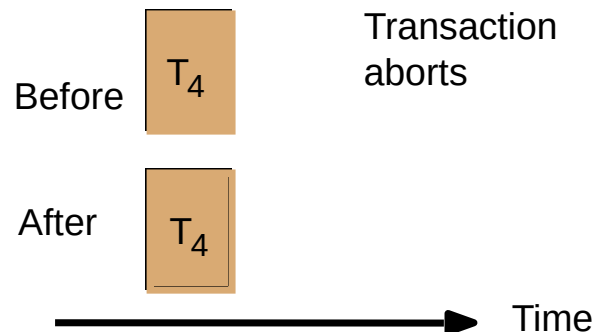


Tentative

(c) T_3 write



(d) T_3 write



object produced
by transaction T_i
(with write timestamp T_i)

$T_1 < T_2 < T_3 < T_4$

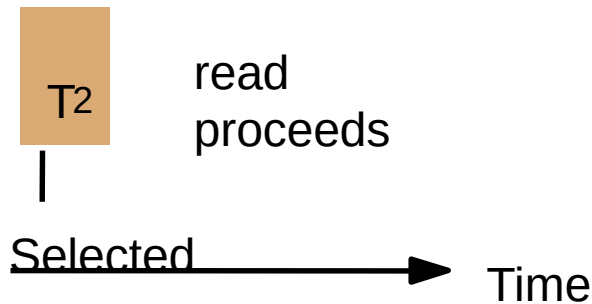
Regra de leitura

Operação de leitura requisitada por transação T_c em objeto D :

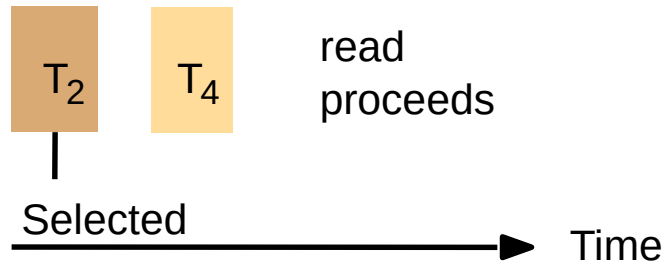
```
if ( $T_c > \text{write\_timestamp}$  on committed version of  $D$ )
{
    let  $D_{\text{selected}}$  be the version of  $D$  with the maximum  $\text{write\_timestamp} \leq T_c$ 
    if ( $D_{\text{selected}}$  is committed)
        perform read operation on the version  $D_{\text{selected}}$ 
    else
        Wait until the transaction that made version  $D_{\text{selected}}$  commits
        or aborts then reapply the read rule
}
else
    Abort transaction  $T_c$ 
```

Exemplo de leitura

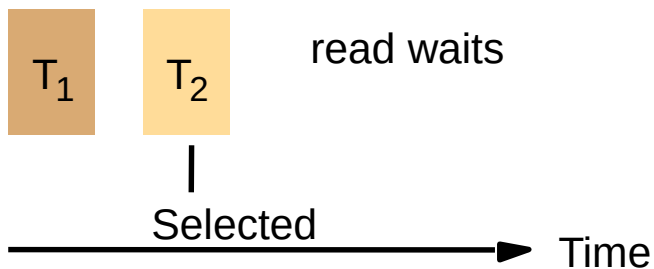
(a) T_3 read



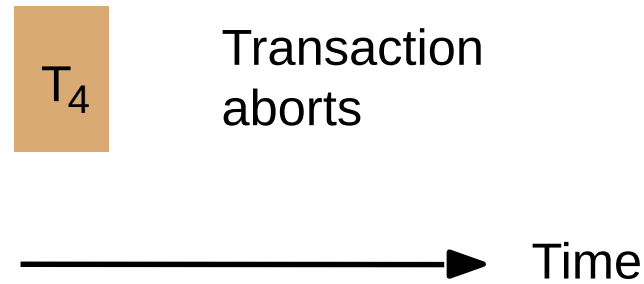
(b) T_3 read



(c) T_3 read



(d) T_3 read



Key:



Committed



Tentative

object produced
by transaction T_i
(with write timestamp T_i)
 $T_1 < T_2 < T_3 < T_4$

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012.
 - Capítulo 16